**TRIBHUVAN UNIVERSITY**

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

# EMBEDDED SYSTEM (CT 655)

## QUESTION BANK WITH ANSWERS

Chapter 7: Control System

### COURSE SYLLABUS AT A GLANCE

| Unit | Topic | Hours | Marks |
|---|---|---|---|
| 1 | Introduction to Embedded System | 3 | 4 |
| 2 | Hardware Design Issues | 4 | 8 |
| 3 | Software Design Issues | 6 | 8 |
| 4 | Memory | 5 | 8 |
| 5 | Interfacing | 6 | 8 |
| 6 | Real-Time Operating System (RTOS) | 8 | 12 |
| 7 | Control System | 3 | 8 |
| 8 | IC Technology | 3 | 8 |
| 9 | Microcontrollers in Embedded Systems | 3 | 8 |
| 10 | VHDL | 4 | 8 |
| | **Total** | **45** | **80** |

**Chapter 7 Topics (3 Hours | 8 Marks):**

- 7.1 Open-Loop Control System — Block Diagram, Characteristics, Examples
- 7.2 Closed-Loop (Feedback) Control System — Block Diagram, Characteristics
- 7.3 Open-Loop vs Closed-Loop Comparison
- 7.4 Proportional (P) Controller — Stability, Steady-State Error
- 7.5 PID Controller — P, I, D Terms, Transfer Function, Block Diagram
- 7.6 PID Effects on System Response (Kp, Ki, Kd impact)
- 7.7 PID Software Algorithm and C Code Implementation
- 7.8 PID Tuning — Ziegler-Nichols Method, Trial and Error
- 7.9 Performance Metrics — Rise Time, Overshoot, Settling Time, Steady-State Error
- 7.10 Control System Applications — Cruise Control, Temperature, Motor Speed

# Chapter 7: Control System

| Q1. Compare and contrast open-loop and closed-loop control systems with block diagrams and examples. | 2079 Chaitra (Regular) \| 2076 Jestha (Back) \| 2073 Bhadra (Regular) [4+4] |
|---|---|

**ANSWER**

**Open-Loop Control System:**

```
┌──────────────┐    u(t)    ┌──────────────┐    y(t)
│  Controller  │──────────▶│  Plant G(s)  │──────────▶ Output
│   C(s)       │            │              │
└──────────────┘            └──────────────┘

  ▲
  │ r(t) Reference/Setpoint

  NO feedback path — output is NOT measured or used for correction
```

**Characteristics of Open-Loop:**

- No feedback: Output is not measured or fed back to input.
- Cannot self-correct: Disturbances (load changes, noise) cause permanent error.
- Simple and cheap: No sensors or feedback circuitry needed.
- Stable by design: Cannot become unstable due to feedback.
- Examples: Toaster (timer-based, no temperature feedback), irrigation timer, washing machine spin cycle timer, stepper motor (open-loop position), traffic lights (time-based).

**Closed-Loop (Feedback) Control System:**

```
          e(t)=r-y                u(t)                y(t)
r(t)──▶[+]──▶[ Controller C(s) ]──▶[ Plant G(s) ]──┬──▶ Output
       [-]◀──────────────────────────────────────┘
               Feedback/Sensor H(s)

  e(t) = r(t) - y(t)    (error = setpoint - actual output)
  Controller minimizes e(t) by adjusting u(t)
```

**Characteristics of Closed-Loop:**

- Feedback: Output continuously measured and compared to setpoint.
- Self-correcting: Automatically compensates for disturbances and plant variations.
- Higher accuracy: Steady-state error can be eliminated (with integral control).
- Can be unstable: Requires careful controller design to ensure stability.
- More complex: Requires sensors, feedback circuitry, and controller tuning.
- Examples: Car cruise control, home thermostat, motor speed regulator, autopilot, blood glucose regulation (insulin pump).

| Aspect | Open-Loop | Closed-Loop |
|---|---|---|
| Feedback | None | Output fed back to input |
| Error Correction | Cannot correct disturbances | Automatically corrects deviations |
| Accuracy | Lower — calibration-dependent | Higher — feedback reduces error |
| Stability | Always stable | Can be unstable if poorly tuned |
| Complexity | Simple, cheap | More complex, needs sensors |
| Sensitivity to disturbances | High — disturbances cause permanent error | Low — controller rejects disturbances |
| Response to plant changes | Cannot adapt | Adapts automatically |
| Cost | Low | Higher (sensors, ADC, controller) |
| Examples | Toaster, timer, stepper motor | Cruise control, thermostat, servo |

| | |
|---|---|
| **Q2. Draw a well-labelled closed-loop control system for a car cruise controller. Derive the condition for a linear proportional controller for stability and to avoid oscillation.** | **2082 Kartik (Back) \| 2080 Ashwin (Back) \| 2075 Baisakh (Back) [2+6]** |

**ANSWER**

**Closed-Loop Cruise Control Block Diagram:**

```
                    e(t)=r-y            u(t)=Kp·e(t)
  r(t) ──────►[+]──────►[ Kp            ]──►[ Throttle  ]──►[ Engine
]──┬──► y(t)
  Desired       [-]         Proportional       Actuator        /Car        │
Actual
  Speed          ▲          Controller                                     │
Speed
                 │                                                         │
                 └──────────────────[  Speed Sensor  ]◄──────────────────┘


  Disturbances (road grade, headwind, load) ──────────────────►[+]──► into
plant


  r(t) = desired speed  |  y(t) = actual speed  |  e(t) = speed error
  u(t) = throttle command  |  Kp = proportional gain
```

**Block components:**

- r(t): Reference speed (set by driver via cruise control stalk).
- Summing junction [+/-]: Computes error e(t) = r(t) − y(t).
- Proportional Controller: Computes throttle command u(t) = Kp × e(t).
- Throttle Actuator: Opens/closes throttle valve proportional to u(t).
- Plant (Engine/Car): Converts throttle opening to vehicle speed y(t).
- Speed Sensor: Measures actual speed y(t) and feeds back to summing junction.

**Proportional Controller — Stability Analysis:**

Model the car as a first-order plant (simplified):

```
Plant transfer function:  G(s) = K / (τs + 1)
  K  = DC gain (speed per unit throttle)
  τ  = time constant (seconds) — inertia of car

Controller: C(s) = Kp

Closed-loop transfer function:
         Kp·G(s)          Kp·K
T(s) = ─────────────── = ──────────────────
       1 + Kp·G(s)       τs + (1 + Kp·K)

Characteristic equation (denominator = 0):
  τs + (1 + Kp·K) = 0
  s = −(1 + Kp·K) / τ
```

**Stability Condition:**

- For stability: pole must be in the LEFT half s-plane → real part of s < 0.
- $s = -(1 + K_p \cdot K)/\tau$ is always negative when $K_p > 0$, $K > 0$, $\tau > 0$.
- Therefore a first-order plant with proportional control is ALWAYS stable for any positive Kp.

**No-Oscillation Condition (Second-Order Plant):**

For a more realistic second-order model (includes actuator dynamics):

```
Plant: G(s) = K·ωn² / (s² + 2ζ□·ωn·s + ωn²)

Closed-loop characteristic equation (with C(s)=Kp):
  s² + 2ζ□·ωn·s + (1 + Kp·K)·ωn² = 0

Effective damping ratio: ζ_eff = ζ□ / √(1 + Kp·K)

No oscillation condition: ζ_eff ≥ 1 (critically or over-damped)
  ζ□ / √(1 + Kp·K) ≥ 1
  ζ□² ≥ 1 + Kp·K
  Kp ≤ (ζ□² − 1) / K

Maximum Kp to avoid oscillation: Kp_max = (ζ□² − 1) / K
```

Note: Proportional control alone always has a steady-state speed error. Adding an Integral term (PI or PID) eliminates this offset.

| Q3. What is a PID controller? Explain each term with its effect on system response. Write the PID equation in time domain and Laplace domain. | 2081 Chaitra (Regular) \| 2079 Chaitra (Regular) \| 2078 Baishakh (Back) [2+6] |
|---|---|

**ANSWER**

### PID Controller:

A PID (Proportional-Integral-Derivative) controller is the most widely used feedback controller in embedded control systems. It computes a control output u(t) based on three terms derived from the error signal e(t) = r(t) − y(t):

```
PID Block Diagram:

                          ┌──────────────────────────────┐
                          │          PID Controller      │
                          │  ┌──────────┐                │
                ┌─────────┼─►│  Kp · e  │──────────┐     │
e(t)            │         │  └──────────┘          │     │
r──►[+]─────────┤         │                        ▼     │          y(t)
    [-]         │         │  ┌──────────┐                │
     ▲          └─────────┼─►│Ki·∫e dt  │──────►[+]──────►│►Plant──►──┬──►
     │                    │  └──────────┘          ▲     │          │
     │                    │                        │     │          │
     │                    │  ┌──────────┐          │     │          │
     │                    │  │Kd·(de/dt)│──────────┘     │          │
     │                    └─►└──────────┘                ┘          │
     └──────────────────────────────────── Feedback ──────────────────┘
```

### PID Equation (Time Domain):

```
u(t) = Kp·e(t)  +  Ki·∫e(t)dt  +  Kd·(de/dt)
         |              |                |
    Proportional    Integral         Derivative
       term           term             term

where: e(t) = r(t) - y(t)  (error = setpoint - measured output)
```

### PID Transfer Function (Laplace Domain):

```
C(s) = Kp  +  Ki/s  +  Kd·s

     = (Kd·s² + Kp·s + Ki) / s
```

### Effect of Each Term:

| Term | Formula | Effect on Response | Trade-off |
|---|---|---|---|
| Proportional (P) | Kp × e(t) | Reduces error magnitude; speeds up response; larger Kp = faster but more overshoot | Cannot eliminate steady-state error (offset remains) |

| | | | |
|---|---|---|---|
| Integral (I) | Ki × ∫e(t)dt | Eliminates steady-state error by accumulating past error; forces final error to zero | Too large Ki causes oscillation (integral windup); slows response |
| Derivative (D) | Kd × de/dt | Predicts future error from rate of change; dampens overshoot; improves stability | Amplifies high-frequency noise; should be used carefully |

| Increase... | Rise Time | Overshoot | Settling Time | Steady-State Error | Stability |
|---|---|---|---|---|---|
| Kp | Decrease | Increase | Small change | Decrease | Degrade |
| Ki | Decrease | Increase | Increase | Eliminate | Degrade |
| Kd | Minor decrease | Decrease | Decrease | No effect | Improve |

| | |
|---|---|
| **Q4. Write the software algorithm for a PID controller. Implement it in C code for an embedded system (e.g., temperature controller).** | **2077 Chaitra (Regular) \| 2073 Bhadra (Back) \| 2074 Bhadra (Back) [4+4]** |

**ANSWER**

**PID Controller Algorithm:**

1. Initialize: Set Kp, Ki, Kd, dt (sampling period). Reset integral=0, prev_error=0, output=0.
2. Read Setpoint: r = desired_value (e.g., 75°C for temperature controller).
3. Read Process Variable: y = measure_output() (e.g., read ADC from thermistor).
4. Compute Error: e = r − y.
5. Proportional Term: P_term = Kp × e.
6. Integral Term: integral = integral + e × dt. I_term = Ki × integral.
7. Derivative Term: D_term = Kd × (e − prev_error) / dt.
8. Compute Output: u = P_term + I_term + D_term.
9. Clamp Output: u = clamp(u, u_min, u_max) to prevent actuator saturation.
10. Anti-Windup: If u is saturated (at limit), stop accumulating integral to prevent windup.
11. Apply Output: send u to actuator (PWM duty cycle, DAC voltage, relay).
12. Update State: prev_error = e. Wait dt seconds. Return to step 2.

**C Code Implementation (Temperature Controller):**

```
#include <stdint.h>

/* PID parameters — tune these for your system */
#define Kp   2.0f
```

```
#define Ki   0.5f
#define Kd   0.1f
#define DT   0.01f    /* sampling period = 10 ms */
#define OUT_MIN  0.0f /* min output: 0% heater PWM */
#define OUT_MAX  100.0f /* max output: 100% heater PWM */

typedef struct {
    float setpoint;      /* desired temperature (°C) */
    float integral;      /* accumulated integral */
    float prev_error;    /* previous error for derivative */
} PID_t;

float PID_Update(PID_t *pid, float measured) {
    /* Step 1: Compute error */
    float error = pid->setpoint - measured;

    /* Step 2: Proportional term */
    float P_term = Kp * error;

    /* Step 3: Integral term (with anti-windup) */
    pid->integral += error * DT;
    float I_term = Ki * pid->integral;

    /* Step 4: Derivative term (on error, not measurement) */
    float D_term = Kd * (error - pid->prev_error) / DT;
    pid->prev_error = error;

    /* Step 5: Compute total output */
    float output = P_term + I_term + D_term;

    /* Step 6: Clamp output to valid range */
    if (output > OUT_MAX) {
        output = OUT_MAX;
        pid->integral -= error * DT; /* anti-windup: undo integral */
    } else if (output < OUT_MIN) {
        output = OUT_MIN;
        pid->integral -= error * DT; /* anti-windup */
    }
    return output;  /* PWM duty cycle 0-100% */
}

/* Usage in main control loop (called every DT = 10ms by timer ISR) */
void Control_Loop(void) {
    static PID_t temp_pid = {.setpoint=75.0f};  /* 75°C target */
    float current_temp = Read_Temperature_ADC(); /* read sensor */
    float pwm = PID_Update(&temp_pid, current_temp);
    Set_Heater_PWM(pwm);  /* apply output */
}
```

| **Q5. Explain different performance metrics of a control system. Illustrate with a step response diagram.** | **2077 Chaitra (Regular) \| 2071 Bhadra (Back) \| 2075 Baisakh (Back) [4+4]** |
|---|---|

## ANSWER

### Performance Metrics of a Control System (Step Response):

```
  Output y(t)
    |
    |                        ┌──── Mp (Peak overshoot)
    |                    ┌──┤
    |                ┌──┘  |  ┌──────────────────── y_ss (steady state)
    |              ┌┘     └──┘   (settling within ±2% band)
    |            ┌─┘
    |          ┌─┘
    |        ┌─┘
    |──────┌─┘
    |──────────────────────────────────────────────────────────► time t
   0      |◄──►|      |◄──────────►|◄──────────────────────►|
          t_d  t_r        t_p              t_s
   (delay)(rise)(      peak)          (settling)

  ess = r - y_ss   (steady-state error — gap between setpoint and final
output)
```

| Metric | Symbol | Definition | Formula/Note |
|---|---|---|---|
| Delay Time | t_d | Time for output to reach 50% of final value | t_d = (1 + 0.7ζ)/ωn (2nd order) |
| Rise Time | t_r | Time for output to rise from 10% to 90% of final value | t_r ≈ 1.8/ωn (underdamped) |
| Peak Time | t_p | Time to reach maximum overshoot peak | t_p = π / (ωn√(1−ζ²)) |
| Maximum Overshoot | Mp | Peak value above steady-state as percentage | Mp = e^(−πζ/√(1−ζ²)) × 100% |
| Settling Time | t_s | Time for output to stay within ±2% of final value | t_s ≈ 4/(ζ·ωn) (2% criterion) |
| Steady-State Error | e_ss | Remaining error after transient dies out | e_ss = lim[s→0] s·E(s) (Final Value Theorem) |

### Effects of Damping Ratio ζ on Response:

- ζ > 1 (Over-damped): Slow, no overshoot. System is sluggish.
- ζ = 1 (Critically damped): Fastest response without overshoot. Ideal for many control applications.

- $0 < \zeta < 1$ (Under-damped): Fast rise but oscillates before settling. Smaller $\zeta \rightarrow$ more oscillation.
- $\zeta = 0$ (Undamped): Continuous oscillation — marginally stable.
- $\zeta < 0$ (Unstable): Unbounded oscillation — system unstable.

| **Q6. Explain PID tuning. Describe the Ziegler-Nichols tuning method.** | **2079 Chaitra (Regular) \| 2080 Ashwin (Back) \| 2078 Jestha (Back) [3+5]** |
|---|---|

**ANSWER**

**PID Tuning:**

PID tuning is the process of selecting the values of Kp, Ki, and Kd to achieve desired system performance (fast response, low overshoot, zero steady-state error, stability). Poorly tuned PID leads to oscillation, slow response, or instability.

**Method 1: Trial and Error (Manual Tuning):**

1. Start with Ki=0, Kd=0. Increase Kp until system responds adequately (fast but slight overshoot).
2. Add Ki slowly to eliminate steady-state error. Watch for increased oscillation.
3. Add Kd to reduce overshoot and improve damping. Increase carefully (noise sensitive).
4. Repeat adjustments iteratively until desired performance is achieved.

**Method 2: Ziegler-Nichols Tuning (Most Common Exam Method):**

Step:

1. Find Ultimate Gain (Ku) and Ultimate Period (Tu):
2. Set Ki = 0 and Kd = 0 (P-only control).
3. Increase Kp slowly from zero.
4. When the system output shows sustained constant-amplitude oscillations (marginally stable), record:
   - Ku = current value of Kp (Ultimate Gain)
   - Tu = period of the sustained oscillations (Ultimate Period) in seconds

**Step 2 — Apply Ziegler-Nichols Formulas:**

| Controller Type | Kp | Ki (= Kp/Ti) | Kd (= Kp×Td) |
|---|---|---|---|
| P only | 0.50 × Ku | — | — |
| PI | 0.45 × Ku | Kp / (0.85 × Tu) | — |
| PID | 0.60 × Ku | Kp / (0.50 × Tu) | Kp × (0.125 × Tu) |

```
Example: System reaches sustained oscillation at Ku=10, Tu=4 seconds
PID gains (Ziegler-Nichols):
  Kp = 0.60 × 10 = 6.0
  Ti = 0.50 × Tu = 2.0 s  →  Ki = Kp/Ti = 6.0/2.0 = 3.0
  Td = 0.125 × Tu = 0.5 s →  Kd = Kp×Td = 6.0×0.5 = 3.0
```

**Method 3: Software Auto-Tuning:**

- Relay feedback method: Automatically finds Ku and Tu using an on/off relay test.
- Model-based tuning: System identification from step response, then optimal PID parameters computed.
- Modern RTOS and microcontrollers (STM32, Arduino PID library) support auto-tuning routines.

**Practical PID Implementation Considerations:**

- Anti-windup: When output saturates (actuator at limit), stop integrating to prevent integral windup — otherwise controller takes long to recover when error reverses.
- Derivative on measurement: Apply derivative to −y(t) instead of e(t) to avoid 'derivative kick' on setpoint step changes.
- Low-pass filter on derivative: D term amplifies noise. Add first-order filter: $K_d \cdot s/(\tau_f \cdot s + 1)$.
- Sampling rate: Sample at least 10–20× faster than the system's bandwidth to avoid aliasing.

| **Q7. Design a control system for an automobile cruise control using open-loop control with proportional control. What are its limitations?** | **2081 Chaitra (Regular) \| 2074 Jestha (Back) [2+6]** |
|---|---|

**ANSWER**

**Open-Loop Cruise Control with Proportional Controller:**

```
Open-Loop System (No Feedback):

  r(t) ──►[ Kp ]──► u(t) ──►[ Throttle Actuator ]──►[ Engine/Car ]──► y(t)
  Desired        Throttle
Actual
  Speed          Command
Speed

  u(t) = Kp × r(t)      (throttle proportional to desired speed)
  No sensor, no feedback — output not measured for control
```

**Design Steps:**

1. Model the plant (car): At steady speed on flat road, throttle u required for speed y is: $y = K \times u$, so $K = y_{rated} / u_{rated}$. Example: 60 km/h requires 30% throttle → K = 2.

2. Set controller gain: Kp = 1/K = 0.5. Then for r=60 km/h: u = 0.5 × 60 = 30% throttle. Correct at the design point.
3. Verify: At nominal flat road and standard load, u = Kp × r gives the correct speed. System works.

**Transfer Function:**

```
Open-loop system TF:  Y(s)/R(s) = Kp × G(s)
For first-order car model G(s) = K/(τs+1):
  Y(s)/R(s) = Kp·K / (τs+1)
Steady-state (s→0):  y_ss = Kp·K·r
For perfect tracking: Kp = 1/K  →  y_ss = r  (at design conditions only)
```

**Limitations of Open-Loop Cruise Control:**

- Cannot correct disturbances: Going uphill increases road load — speed drops and controller cannot detect or correct it (no sensor). The car slows down permanently.
- Calibration-dependent: Kp is set for one specific road condition (flat road, standard load, no wind). Any change causes error.
- No speed measurement: System doesn't know actual speed — only applies throttle based on desired speed.
- Load variation: Adding passengers or cargo changes the plant gain K — recalibration needed.
- Temperature and fuel variation: Engine efficiency changes with temperature, affecting K.
- Conclusion: Open-loop proportional control is only accurate at one operating point. Real cruise control systems are closed-loop (feedback) with PID controllers to handle all these disturbances automatically.

| **Q8. Design a PID controller for closed-loop cruise control. Explain how each PID term helps achieve the desired speed accurately.** | **2082 Kartik (Back) \| 2080 Chaitra (Regular) \| 2079 Jestha (Back) [8]** |
|---|---|

**ANSWER**

**PID Closed-Loop Cruise Control:**

```
                e=r-y                      u(t)                y(t)
  r(t) ──►[+]──────►[ PID Controller ]──►[ Throttle ]──►[ Car ]──┬──►
Speed
            [-]◄──────────────────────────────────────────────────┘
                                      Speed Sensor

  e(t) = r(t) - y(t)    (speed error in km/h)
  u(t) = Kp·e + Ki·∫e dt + Kd·(de/dt)    (throttle command 0-100%)
```

**Role of Each PID Term in Cruise Control:**

**P term (Kp·e):** When the car is below desired speed (e > 0), P term opens throttle proportionally to the error. Larger error → more throttle. Gets the car close to speed quickly. But P alone leaves a residual speed error (steady-state error).

**I term (Ki·∫e dt):** Accumulates all past errors over time. If the car is slightly slow for a long time (going uphill), the integral grows, pushing more throttle until speed error reaches zero. Eliminates steady-state speed error completely. Without I, the car would always be slightly slower than desired on hills.

**D term (Kd·de/dt):** Detects how quickly the error is changing. If a gust of wind suddenly drops speed (e rising rapidly), D term gives extra throttle immediately, before the car slows significantly. Also prevents overshoot — if car is approaching desired speed rapidly (e decreasing fast), D term reduces throttle to avoid overshooting.

**Design Steps:**

1. Model plant: $G(s) = K/(\tau s + 1)$, identify K and τ from step test on vehicle.
2. Choose Kp using Ziegler-Nichols or simulation: gives fast response.
3. Add Ki to eliminate steady-state error on hills and with load changes.
4. Add Kd to reduce overshoot when speeding up to target.
5. Implement in firmware with dt = 50ms timer interrupt (control loop at 20 Hz).
6. Add anti-windup on integral (when throttle saturated at 0% or 100%).
7. Test: flat road, uphill, downhill, sudden load change — verify speed tracks setpoint.

**Typical Tuned Values (example):**

```
Kp = 3.0  (proportional — responsive to speed error)
Ki = 0.5  (integral — eliminates uphill steady-state error over ~2
seconds)
Kd = 0.8  (derivative — damps overshoot when accelerating to target speed)
dt = 50ms (control loop runs every 50ms = 20 Hz)
```

| **Q9. Explain stability of a control system. Explain the conditions for no unbounded output and no oscillation for a closed-loop system.** | **2075 Baisakh (Back) \| 2073 Bhadra (Regular) \| 2076 Chaitra (Back) [4+4]** |
|---|---|

**ANSWER**

**Control System Stability:**

A control system is stable if, for any bounded input (including a step), the output remains bounded and eventually settles to a steady state. An unstable system's output grows without bound.

### Stability in Terms of Poles:

```
Closed-loop transfer function: T(s) = N(s) / D(s)
Poles = roots of characteristic equation D(s) = 0

s-plane analysis:
  jω
   |  Unstable     |  Stable
   |  (poles here)|  (poles here)
──┼──────────────┼──────────── σ (real axis)
   |   Re(s) > 0   |  Re(s) < 0
   |               |

BIBO Stability: System is stable ↔ ALL poles are in LEFT half s-plane
(Re(s) < 0)
```

### Condition for No Unbounded Output (BIBO Stability):

- All roots of the characteristic equation D(s) = 0 must have strictly negative real parts.
- Equivalently: all poles of T(s) must lie in the open left half of the s-plane.
- Check: Routh-Hurwitz criterion — all coefficients of the characteristic polynomial must be positive AND all elements in first column of Routh array must be positive.

### Condition for No Oscillation (Non-Oscillatory Response):

- All poles of T(s) must be real (no complex conjugate pairs).
- For a second-order system $T(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$:

```
Poles: s = −ζωn ± ωn√(ζ²−1)

Poles are REAL when:  ζ² − 1 ≥ 0  →  ζ ≥ 1

No oscillation condition: ζ ≥ 1  (critically or over-damped)

For proportional control of plant G(s) = K/(τs+1):
  Effective ζ = (1 + KpK) / (2·ωn·τ)
  No oscillation: (1 + KpK)² ≥ 4·KpK·τ·ωn²
  →  Kp_max = (ζ□² − 1)/K  (maximum Kp before oscillation begins)
```

### Summary of Design Conditions:

| Condition | Mathematical Requirement | Physical Meaning |
|---|---|---|
| Stability (no blow-up) | All poles: Re(s) < 0 | System output stays bounded for all time |
| No oscillation | All poles real: $\zeta \geq 1$ | Step response rises smoothly without ringing |

| | | |
|---|---|---|
| Zero steady-state error | System type ≥ 1 (integrator in loop) | Add I term in PID controller |
| Fast response | Poles far left (large \|Re(s)\|) | Large ωn — but conflicts with stability margin |

| | |
|---|---|
| **Q10. Design a closed-loop temperature control system for an embedded heater. Explain how PID is applied and what sensors/actuators are used.** | **Model Question \| 2076 Baisakh (Back) [4+4]** |

**ANSWER**

**Embedded Temperature Control System:**

```
┌──────────────────────────────────────────────────────────────────┐
│                EMBEDDED TEMPERATURE CONTROLLER                   │
│                                                                   │
│  Setpoint    e(t)        PID           PWM          Heater       │
│  (75°C) ──►[+]──►[ MCU PID   ]──►[ PWM Gen ]──►[ Element ]──┐│
│             [-]    Algorithm                     (SSR relay)   ││
│              ▲                                                  ││
│              │        ADC           Temp                       ││
│              └──────[MCU ADC ]◄──[Sensor   ]◄──────────────────┘│
│                                  (Thermistor/                   │
│                                   PT100/DS18B20)                │
└──────────────────────────────────────────────────────────────────┘
```

**Hardware Components:**

- Temperature Sensor: Thermistor (NTC — resistance decreases with temperature), PT100 (RTD), or DS18B20 (digital sensor, 12-bit, ±0.5°C accuracy).
- ADC: MCU's built-in 12-bit ADC reads thermistor voltage divider output and converts to temperature.
- MCU (e.g., STM32): Runs PID algorithm in a timer ISR every 100ms (10 Hz control loop).
- PWM Output: MCU generates PWM signal (0–100% duty cycle) representing heater power command u(t).
- SSR (Solid State Relay): Switches mains AC heater element ON/OFF at the PWM rate (zero-crossing switching).
- Heater Element: Resistive heating element — power proportional to PWM duty cycle.

**PID Applied to Temperature Control:**

- P term: If temperature is 10°C below setpoint, apply proportional heating power. Reduces gap quickly.
- I term: If oven is consistently 2°C below setpoint (heat loss through door seal), integral accumulates and increases duty cycle until offset is eliminated.

- D term: When temperature is rising rapidly toward setpoint, derivative reduces power early to avoid overshoot past 75°C. Prevents thermal runaway.

```
Typical PID parameters for temperature controller (slow thermal system):
  Kp = 5.0     (moderate gain — temperature systems are slow)
  Ki = 0.1     (slow integration — thermal lag is large)
  Kd = 2.0     (significant derivative — dampen thermal overshoot)
  dt = 100ms   (10 Hz control loop — sufficient for slow thermal dynamics)
```

| **Q11. Compare P, PI, PD, and PID controllers. When would you choose each?** | **Model Question \| 2074 Chaitra (Back) [4+4]** |
|---|---|

**ANSWER**

| Controller | Transfer Function C(s) | Eliminates SS Error? | Handles Noise? | Best Used When |
|---|---|---|---|---|
| P | Kp | No (offset remains) | Yes | Simple systems, some steady-state error acceptable |
| PI | Kp + Ki/s | Yes | Yes (moderate) | Most industrial processes — eliminates offset, moderate speed |
| PD | Kp + Kd·s | No | No (amplifies noise) | Fast systems with noise-free measurement, no offset needed |
| PID | Kp + Ki/s + Kd·s | Yes | Moderate (filter D) | Standard choice for most embedded control applications |

**When to Choose Each:**

**P only:** Simple position control where small steady-state error is acceptable (e.g., basic motor speed, proportional valve). Fast, no integrator windup risk.

**PI:** Most common industrial choice. Use when steady-state error must be zero but measurement noise makes derivative impractical. Examples: flow control, level control, most process control.

**PD:** Use when fast response and damping are needed but there is no steady-state error requirement. Example: servo position systems where mechanical stops define final position.

**PID:** Standard choice for most embedded control systems — cruise control, temperature, motor speed, pressure control. When steady-state accuracy and good transient response both required. Use derivative filter if sensor noise is present.

# Summary: Topics Covered in Chapter 7

| S.N. | Topic | Exam(s) Asked | Marks |
|---|---|---|---|
| 1 | Open-Loop vs Closed-Loop Comparison (table + diagrams) | 2079 Chaitra, 2076 Jestha, 2073 Bhadra (annual) | 4+4 |
| 2 | Closed-Loop Cruise Control Block Diagram (full labels) | 2082 Kartik, 2080 Ashwin, 2075 Baisakh (annual) | 2+6 |
| 3 | Proportional Controller — Stability + No-Oscillation Condition | 2082 Kartik, 2080 Ashwin, 2075 Baisakh (annual) | 4–6 |
| 4 | PID Controller — Definition, Block Diagram, Equations | 2081 Chaitra, 2079 Chaitra, 2078 Baishakh (annual) | 2+6 |
| 5 | PID Effect Table (Kp/Ki/Kd effect on rise time, overshoot etc.) | 2081 Chaitra, 2079 Chaitra, 2077 Chaitra (annual) | 4 |
| 6 | PID Software Algorithm (12 steps) | 2077 Chaitra, 2073 Bhadra, 2074 Bhadra (annual) | 4 |
| 7 | PID C Code Implementation (full embedded C with anti-windup) | 2077 Chaitra, 2073 Bhadra (annual) | 4 |
| 8 | Performance Metrics (6 metrics — rise time, overshoot, settling, ess) | 2077 Chaitra, 2071 Bhadra, 2075 Baisakh | 4+4 |
| 9 | PID Tuning — Trial and Error + Ziegler-Nichols method (table) | 2079 Chaitra, 2080 Ashwin, 2078 Jestha | 3+5 |
| 10 | Open-Loop Cruise Control with P — Design + Limitations | 2081 Chaitra, 2074 Jestha | 2+6 |
| 11 | Full PID Closed-Loop Cruise Control Design | 2082 Kartik, 2080 Chaitra, 2079 Jestha | 8 |
| 12 | Stability Conditions (BIBO + no-oscillation + Routh-Hurwitz) | 2075 Baisakh, 2073 Bhadra, 2076 Chaitra | 4+4 |
| 13 | Temperature Controller Design (sensors, actuators, PID parameters) | 2076 Baisakh, Model Q | 4+4 |
| 14 | P vs PI vs PD vs PID — Comparison Table + Selection Guide | 2074 Chaitra, Model Q | 4+4 |